

Çalışma Zamanı Uygulama Öz Koruması (RASP)

CEN429 Güvenli Programlama — Hafta 6

Dr. Öğr. Üyesi Uğur CORUH · 23.10.2026

Bugünün planı (3 saat)

Saat	Bölüm	Konu
1	0–2	Temel kavramlar · RASP nedir · RASP mimarisi
2	3–7	Bütünlük · hata ayıklayıcı · emülatör · kanca/Frida · bellek koruması
3	8–13	Kök/imza · kontrol akışı bütünlüğü · tepki politikası · sınırlar · proje

Bu hafta nereye oturuyor?

- 4–5. hafta: kodu **statik** sağladık (derleme, gizleme).
- Bu hafta (6): program **çalışırken** kendini korur → RASP.
- 9/11/14: gizleme ve whitebox RASP ile birlikte çalışır.

Öğrenme çıktısı

Bu hafta **ÖÇ.3** üstünedir.

Sonunda yapabileceğiniz:

- RASP denetimlerini (bütünlük, anti-debug, ortam, kanca) tanımak
- Tepki politikası ve cihaz bağlama tasarlamak
- RASP'in **sınırlarını** ve etik çerçevesini açıklamak

Ana fikir

RASP = **Runtime Application Self-Protection**: uygulamanın çalışırken kendini **izlemesi** ve tehdide **tepki** vermesi.

Algıla → savun → caydır. Ama tek denetim değil, **katman**.

0. Temel kavramlar (sıfırdan)

Çalışma anı (runtime) nedir?

- **Çalışma anı:** programın **çalıştığı** an (derleme değil).
- RASP korumaları burada devreye girer: program kendini **çalışırken** izler.

RASP nedir?

- **RASP (Runtime Application Self-Protection):** uygulamanın çalışırken kendini koruması.
- Kurcalama, hata ayıklama, sahte ortam gibi tehditleri **algılar** ve **tepki** verir.

Hata ayıklayıcı (debugger)

- **Debugger:** programı adım adım çalıştırıp durduran, belleği okuyan araç (gdb, lldb).
- Saldırgan bununla akışı izler, değer değiştirir.
- RASP "bana debugger bağlı mı?" diye bakar.

Emülatör ve sanal makine

- **Emülatör/VM:** bir cihazı **taklit eden** yazılım ortamı (Android emülatör, QEMU).
- Saldırgan analizini gerçek cihaz yerine burada yapar (daha kolay).
- RASP sahte ortamı sezmeye çalışır.

Kanca (hook) ve enstrümantasyon

- **Kanca (hook):** bir fonksiyonun çağrısını **araya girip** değiştirme.
- **Enstrümantasyon:** çalışan programa kod enjekte edip davranışını izleme/değiştirme.
- Araç: **Frida** (çok yaygın dinamik enstrümantasyon aracı).

LD_PRELOAD

- **LD_PRELOAD:** Linux'ta bir kütüphaneyi programdan **önce** yükletip fonksiyonları değiştirme yolu.
- Saldırgan bununla kritik fonksiyonları **kancalayabilir**.
- RASP bunu tespit etmeye çalışır.

Bütünlük ve self-hashing

- **Bütünlük (integrity):** kodun/dosyanın **değişmemiş** olması.
- **Self-hashing:** programın **kendi kodunun** özetini hesaplayıp beklenenle karşılaştırması.
- Kurcalanmışsa özet tutmaz.

Özet (checksum/hash)

- **Özet:** bir veriden hesaplanan sabit boyutlu **parmak izi** (SHA-256).
- Veri değişirse özet değişir.
- Bütünlük denetiminin temeli.

Kök (root) / jailbreak

- **Kök (root):** cihaz üzerinde tam yetki (normalde kısıtlı).
- Köklü cihazda korumalar zayıflar; saldırgan her şeye erişir.
- RASP "cihaz köklü mü?" diye bakar.

İmza doğrulama

- Uygulama paketleri **dijital imzayla** imzalanır.
- **İmza doğrulama:** çağıran/yüklenen bileşenin imzasının beklenen olup olmadığını denetleme.
- Sahte/değiştirilmiş bileşeni yakalar.

Kontrol akışı bütünlüğü (sayaç)

- Kritik denetimler **tek bir if** ile yapılırsa, tek nokta yamayla atlanır.
- **Kontrol akışı sayacı**: denetimlerin doğru **sırayla** geçtiğini sayarak doğrulama.
- Tek yama yetmez hale gelir.

Tepki politikası (response)

- **Tepki politikası:** RASP bir tehdit görünce **ne yapacak?**
- Sessizce kapan, işlevi kısıtla, sunucuya bildir, gecikmeli tepki...
- "Hemen çök" her zaman en iyisi değildir.

Cihaz bağlama ve caydırma

- **Cihaz bağlama:** verilerin/anahtarların yalnız **belirli cihazda** anlamlı olması.
- **Caydırma (deterrence):** saldırıyı zahmetli/riskli kılıp vazgeçirme.
- RASP'in nihai amacı: maliyeti yükseltmek.

Şimdi hazırız

Terimler:

çalışma anı · RASP · debugger · emülatör/VM · hook/Frida · LD_PRELOAD · bütünlük/self-hashing ·
özet · kök · imza doğrulama · kontrol akışı sayacı · tepki politikası · cihaz bağlama · caydırma

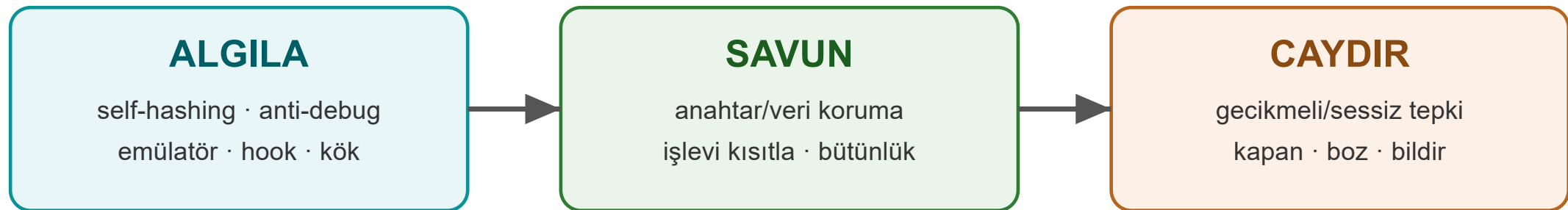
Şimdi: RASP nedir, ne yapar?

1. RASP nedir?

Üç iş: algıla, savun, caydır

- **Algıla:** anormal ortamı/davranışı sez (debugger, emülatör, kurcalama).
- **Savun:** tehdit varsa işlevi kısıtla/durdur.
- **Caydır:** saldırıyı zahmetli kıl → saldırgan vazgeçsin.

RASP döngüsü — algıla → savun → caydır



Tek denetim değil, birbirini kontrol eden örtüşen ağ → tek yama yetmez.

RASP neyi tamamlar?

- Statik korumalar (4–5. hafta) **çalışmadan** geçerli.
- Gizleme (9/11/14) **anlamayı** zorlaştırır.
- RASP, program **çalışırken** aktif savunma ekler.

Gizleme RASP'ı gizler; RASP gizlemeyi çalışırken korur.

RASP neden gerekli?

- Beyaz kutu saldırgan programı **çalıştırıp** izler (debugger, Frida).
- Statik gizleme bunu tek başına durdurmaz.
- RASP "beni izliyorlar mı?" diye bakıp tepki verir.

Ana kural (baştan)

RASP tek başına aşılmaz değildir; **katmanlı** ve **çeşitli** denetimlerle güç kazanır.

Tek bir "root mu?" denetimi kolayca atlanır; onlarca farklı denetim birlikte zorlaşır.

2. RASP mimarisi

Denetim ne zaman çalışır?

- **Başlangıçta:** açılışta ortam denetimi.
- **Kritik işlem öncesi:** ödeme/anahtar kullanımından hemen önce.
- **Periyodik:** arka planda düzenli.
- **Rastgele:** öngörülemez anlarda (saldırgan zamanlayamasın).

Denetim nerede çalışır?

- **Native tarafta** (C/C++): tersine mühendisliği zor, tercih edilir.
- **Yönetilen tarafta** (Java): daha kolay okunur, ama gerekli.
- **Çapraz denetim**: iki taraf birbirini denetler.

Çapraz denetim fikri

- Java tarafı, native kütüphanenin **özetini** denetler.
- Native taraf, Java tarafının **bütünlüğünü** denetler.
- Saldırgan **ikisini birden** atlamak zorunda.

Denetim nasıl çalışır? (gizli)

- Denetim kodu **gizlenir** (9. hafta): saldırgan denetimi kolayca bulamasın.
- Sonuç **opak** (düz true/false değil).
- Tepki **gecikmeli** olabilir (birazdan).

Katmanlı denetim ağı

- Tek denetim değil, **onlarca** farklı denetim.
- Birbirini denetleyen **ağ** (checker network).
- Birini atlamak diğerini tetikler.

Mimari · özet şema

[başlangıç] → ortam denetimi
[kritik işlem] → bütünlük + anti-debug + hook denetimi
[periyodik/rastgele] → tekrar denetim
↳ hepsi gizli, opak sonuç, çapraz denetim

Bölüm 1–2 — kısa sinama

1. RASP'in üç işi?
2. Çapraz denetim neden güçlü?
3. Neden tek denetim değil, ağ?

Bölüm 1–2 — cevaplar

1. **Algıla** (kurcalama/debug/emülatör), **Savun** (anahtar/veri koruma, işlevi kısıtla), **Caydır** (tepki: kapan/boz/bildir).
2. Denetimler **birbirini** kontrol eder; saldırgan birini kapatınca diğeri fark eder → tek noktayı susturmak yetmez.
3. Tek denetim **tek yamayla** atlanır; örtüşen ağ her düğümü ayrı kırmayı gerektirir → **saldırı potansiyelini** yükseltir.

3. Bütünlük denetimi (self-hashing)

Fikir

- Program, **kendi kod bölümlerinin** özetini hesaplar.
- Beklenen özetle karşılaştırır.
- Kurcalanmışsa özet **tutmaz** → tepki.

Nasıl çalışır? · adım adım

1. Derleme sonrası, kritik kod bloklarının özeti **kaydedilir**.
2. Çalışırken program aynı blokların özetini **yeniden** hesaplar.
3. Kayıtlı özetle karşılaştırır.
4. Fark varsa → kurcalama → tepki.

Neyi yakalar?

- İkili dosyaya yapılan **yama** (patch).
- Bir denetimi atlamak için değiştirilen baytlar.
- Enjekte edilmiş kod.

Self-hashing tek başına yeterli değil

- Saldırgan, **özet fonksiyonunu** bulup her zaman "tutuyor" dedirtebilir.
- Ya da donanım kesme noktalarıyla denetimi atlar (van Oorschot vd.).
- Bu yüzden: gizle, çeşitlendir, çapraz denetle.

Güçlendirme

- Özet denetimi kodu **gizli** (9. hafta).
- **Birden çok** blok, örtüşen bölgeler.
- Sonuç düz karşılaştırma değil, **hesaba karışır** (denetim sonucu programın davranışını etkiler).

Örtüşen denetim

- Bir denetim başka bir denetimin kodunu da kapsar.
- Saldırgan birini kapatınca diğeri **bozulur**.
- Tek noktadan atlamak zorlaşır.

4. Hata ayıklayıcı algılama

Neden debugger tehlikeli?

- Saldırgan programı **durdurup** belleği okur, değer değiştirir.
- Denetimleri tek tek atlayabilir.
- RASP "bana debugger bağlı mı?" diye bakar.

Algılama yolları (kavram)

- İşletim sisteminin "izleniyorum mu?" bilgisini sorma.
- Belirli hata ayıklama arayüzlerinin durumu.
- Zamanlama: bir işlem beklenenden **çok uzun** sürdüyse, biri adım adım çalıştırıyor olabilir.

Zamanlama tabanlı sezme

```
t0 = şimdi  
    ... küçük bir iş ...  
t1 = şimdi  
if (t1 - t0 > eşik) → biri durdurup inceliyor olabilir
```

Debugger adımlaması işi **yavaşlatır**.

Anti-debug atlanabilir

- Her tek yöntem bilinir ve atlanabilir.
- Güç: **birçok** farklı yöntem + gizleme + çeşitlendirme.
- Amaç yine **caydırma** ve **gecikme**.

Tepki nasıl?

- Hemen çökme her zaman iyi değil (saldırgana "burada denetim var" der).
- Daha iyi: **gecikmeli**, **dolaylı** tepki (birazdan bölüm 10).

Bölüm 3–4 — kısa sinama

1. Self-hashing neyi yakalar?
2. Neden tek başına yetmez?
3. Zamanlama nasıl debugger'ı ele verir?

Bölüm 3–4 — cevaplar

1. **Kod/ikili baytlarının değişmesini** (yama, breakpoint/kod enjeksiyonu) — çalışırken kendi özetini hesaplayıp beklenenle karşılaştırır.
2. Saldırgan hash rutinini/beklenen değeri bulup **atlar** veya karşılaştırmayı NOP'lar; ayrıca yalnız **statik** değişikliği yakalar. Örtüşen denetim gerek.
3. Debugger/kesme noktası kodu **yavaşlatır**; iki nokta arası süre eşiği aşarsa hata ayıklama sezilir.

5. Ortam algılama: emülatör/VM

Neden emülatör?

- Saldırgan analizini gerçek cihaz yerine **emülatörde** yapar.
- Emülatörde durdurmak, izlemek, sıfırlamak kolaydır.
- RASP "gerçek cihazda mıyım?" diye bakar.

Algılama ipuçları (kavram)

- Emülatöre özgü donanım/sensör eksiklikleri.
- Tipik sahte tanımlayıcılar.
- Gerçek cihazda beklenen özelliklerin yokluğu.

Emülatör algılama yanlış pozitif verir

- Bazı gerçek cihazlar emülatör gibi görünebilir.
- Geliştiriciler de emülatör kullanır.
- Tepki **sert olmamalı**: kısıtla, engelleme, dikkatle.

6. Kanca ve enstrümantasyon algılama

Kanca (hook) tehdidi

- Saldırgan kritik bir fonksiyonu **kancalar**: çağrıyı araya girip değiştirir.
- Örnek: "imza geçerli mi?" fonksiyonunu her zaman "evet" döndürtmek.
- Araç: **Frida**, Xposed.

LD_PRELOAD kancası

- Linux'ta bir kütüphaneyi programdan önce yükletip fonksiyonları **değiştirme**.
- Kritik fonksiyon saldırganın sürümüyle değişir.
- RASP yüklenen kütüphaneleri denetler.

Algılama yolları (kavram)

- Beklenmeyen yüklenmiş kütüphaneler/modüller.
- Bilinen enstrümantasyon araçlarının izleri (bellekte, portlarda).
- Fonksiyon adreslerinin beklenenden **sapması**.

Fonksiyon bütünlüğü

- Kritik fonksiyonun ilk baytları beklenen mi?
- Kanca genelde başa bir **atlama** yerleştirir → baytlar değişir.
- Değişmişse → kanca var.



Kanca algılama da atlanabilir

- Gelişmiş araçlar izlerini gizler.
- Güç: çoklu yöntem + gizleme + sunucu tarafı denetim.

7. Dinamik bellek koruması

Sorun

- Hassas değerler (anahtar, sayaç) bellekte durur.
- Saldırgan belleği okuyup/değiştirip denetimi atlayabilir.

Korunan sayaç fikri

- Kritik bir sayacı düz tutmak yerine:
 - bir **gölge kopya** + **özet** ile birlikte tut.
 - her okuma/yazmada tutarlılığı denetle.

Korunan değer · kavram

```
typedef struct { uint32_t deger; uint32_t golge; uint32_t ozet; } Korunan;  
/* okuma: deger == ~golge ve ozet doğru mu? */
```

Saldırgan yalnız `deger` 'i değiştirirse tutarsızlık yakalanır.

Bellek izleme tespiti

- Hassas bölgelere beklenmedik erişim izleniyor mu?
- Bellek dökümü/tarama izleri.
- Amaç: canlı bellek analizi maliyetini artırmak.



Bellek koruması sınırlı

- Kararlı saldırgan yine erişebilir.
- Amaç: **geciktirmek**, anahtarı kısa ömürlü kılmak (10. hafta).

Bölüm 5–7 — kısa sinama

1. Emülatör algılamada neden yanlış pozitif riski var?
2. Kanca (hook) fonksiyon baytlarını neden değiştirir?
3. Korunan sayaç neyi yakalar?

Bölüm 5–7 — cevaplar

1. Meşru kullanıcı da **VM/emülatör/CI** kullanabilir; zayıf sinyaller gerçek cihazda da görülür → **suçsuzu engelleme** riski. Sertliği derecelendir.
2. Hook, hedefin **ilk baytlarını** bir atlama (jmp/trampoline) ile değiştirir ki çağrı saldırgan koduna sapsın → **prolog baytları** beklenenden farklı olur.
3. **Kontrol akışının atlanmasını**: kritik denetimler çalıştıkça sayaç artar; beklenen değere ulaşmazsa bir denetim **baypas** edilmiştir.

8. Kök ortam ve imza doğrulama

Köklü cihaz neden risk?

- Kök (root) = tam yetki.
- Korumalar zayıflar; saldırgan belleğe, dosyaya, sürece erişir.
- RASP "cihaz köklü mü?" diye bakar.

Kök algılama (kavram)

- Bilinen kök araçlarının/dosyalarının varlığı.
- Normalde kısıtlı olan işlemlerin yapılabilmesi.
- Sistem bütünlüğü göstergeleri.

Kök algılama atlanabilir

- Kök gizleyiciler (root hiding) vardır.
- Tek gösterge yetmez; **birçok** gösterge + sunucu denetimi.
- Tepki: kritik işlevi kısıtla, tümünden engelleme (kullanıcı mağdur olmasın).

Bileşen imza doğrulaması

- Çağırılan/yüklenen bileşenin **dijital imzası** beklenen mi?
- Sahte/değiştirilmiş bir uygulama kütüphaneyi çağırıyorsa yakala.
- Örnek: yalnız beklenen imzalı ana uygulama SDK'yı kullanabilsin.

İmza + özet birlikte

- İmza: bileşen **kim** tarafından yayımlandı?
- Özet: bileşen **değişti mi**?
- İkisi birlikte sahtekârlığı zorlaştırır.

9. Kontrol akışı bütünlüğü

Tek `if` neden yetmez?

```
if (imza_gecerli()) devam(); /* tek nokta */
```

- Saldırgan bu tek dalı **yamalar** → denetim atlanır.
- ix. haftadaki opak boolean + rastgele çıkış burada kritik.

Kontrol akışı sayacı

- Kritik denetimler doğru **sırayla** geçilmeli.
- Her denetim bir **sayacı** günceller.
- Sonda sayaç beklenen değilse → biri atlandı.

Çift sayaç

- İki bağımsız sayaç (biri artan, biri azalan).
- Toplamları/ilişkileri sabit olmalı.
- Saldırgan **ikisini birden** tutarlı değiştirmek zorunda.

Sonucu davranışa bağla

- Denetim sonucu düz bir `if` değil.
- Sonuç, bir sonraki adımın **hesabına** girer (ör. bir anahtar türetmesine).
- Atlanırsa program **yanlış** çalışır (sessizce bozulur).

10. Tepki politikası ve caydırma

"Hemen çök" neden kötü?

- Saldırgana "işte tam burada denetim var" der.
- Denetimi bulup atlamayı kolaylaştırır.
- Daha akıllı tepkiler gerekir.

İyi tepki örnekleri

- **Gecikmeli:** tehdidi sonra, başka yerde tetikle (kaynağı gizle).
- **Dolaylı:** işlevi sessizce bozar (yanlış sonuç üret).
- **Sunucuya bildir:** risk skoru artır, işlemi sunucuda reddet.

Tepki tasarımı

- Kullanıcı deneyimini bozma (yanlış pozitif riski).
- Sert tepki yalnız **yüksek güvenli** göstergede.
- Çoğu durumda: kısıtla + sunucu denetimi.

Cihaz bağlama

- Anahtar/veri yalnız **belirli cihazda** anlamlı.
- Kopyalanırsa (başka cihaz) işe yaramaz.
- Kod taşımaya (9/11. hafta) karşı.

Caydırma · nihai amaç

RASP saldırıyı **imkânsız** kılmaz; **zahmetli ve riskli** kılar.

Yeterince katman + sunucu denetimi → saldırgan vazgeçer ya da yakalanır.

Bölüm 8–10 — kısa sinama

1. Köklü cihaz neden risk?
2. Tek `if` 'i sayaç neden güçlendirir?
3. Neden "hemen çök" kötü bir tepki olabilir?

Bölüm 8–10 — cevaplar

1. **Root/jailbreak** bellek okuma/yazma, hook, sertifika enjeksiyonu engellerini kaldırır → RASP varsayımları ve izolasyon çöker.
2. Tek **if** bir NOP/yama ile atlanır; **sayaç** denetimin gerçekten çalıştığını sonradan doğrular (dağıtık kanıt) → tek yama yetmez.
3. "Hemen çök" saldırılarına denetimin **tam yerini** gösterir ve yanlış pozitifte meşru kullanıcıyı vurur. **Gecikmeli/sessiz/aldatıcı** tepki daha iyi.

11. Sınırlar ve etik

RASP'in sınırları

- Her tek denetim **atlanabilir**.
- Köklü cihazda güç azalır.
- Yanlış pozitif → gerçek kullanıcı mağdur olabilir.
- Bakım maliyeti yüksek (araçlar gelişir).

O yüzden RASP...

- Katmanlı (çok denetim), çeşitlendirilmiş, gizli.
- Sunucu tarafı denetimle birlikte (asıl güvence).
- Tek başına "aşılmaz" iddiası yanlıştır.

Etik ve kullanıcı

- RASP kullanıcının cihazında çalışır; **aşırı veri toplama** etik değil.
- Yanlış pozitif kullanıcıyı hizmetten edebilir → dikkatli tepki.
- Şeffaflık ve gizlilik dengesi.

Güvenli örnek kuralı

- Ders demoları öğrenci cihazına **zarar vermez**.
- Gerçek kök/sistem değişikliği yok; sentetik göstergeler.
- Amaç kavramı öğretmek, saldırı aracı yapmak değil.

12. Proje: bu hafta (S10)

Proje · S10 (RASP + tepki)

- [] En az iki farklı RASP denetimi (ör. bütünlük + anti-debug).
- [] Denetimlerin **ne zaman/nerede** çalıştığını belgele.
- [] Tepki politikası: tehdit görülünce ne olur?
- [] Cihaz bağlama kararı ve gerekçesi.
- [] Sınır ve kalan risk (neyi korumaz).

Değerlendirici gözüyle

- "RASP var" yetmez; kaç katman, nasıl gizli, nasıl tepki?
- Sunucu denetimiyle nasıl birleşiyor?
- Yanlış pozitif riski nasıl yönetiliyor?

Çözümlü kendini sinama

Soru 1

RASP'in üç işi nedir?

Cevap: Algıla (anormal ortam/kurcalama), savun (işlevi kısıtla/durdur), caydır (maliyeti yükselt).

Soru 2

Self-hashing neyi yakalar, neden tek başına yetmez?

Cevap: İkiliye yapılan yamayı/kurcalamayı yakalar. Saldırgan özet fonksiyonunu bulup atlatabildiği için tek başına yetmez; gizleme + çapraz + örtüşen denetim gerekir.

Soru 3

Çapraz denetim neden güçlüdür?

Cevap: Java native'i, native Java'yı denetler; saldırgan **ikisini birden** atlamak zorunda kalır.

Soru 4

Zamanlama tabanlı anti-debug nasıl çalışır?

Cevap: Küçük bir iş beklenenden çok uzun sürdüyse, biri programı adım adım çalıştırıyor (debugger) olabilir.

Soru 5

Tek **if** denetimini kontrol akışı sayacı neden güçlendirir?

Cevap: Denetimlerin doğru sırayla geçtiğini sayar; tek bir dalı yamalamak sayacı bozar, atlanma yakalanır.

Soru 6

"Hemen çök" neden her zaman iyi bir tepki değildir?

Cevap: Saldırgana denetimin tam yerini gösterir. Gecikmeli/dolaylı tepki + sunucu bildirimi daha iyidir.

Soru 7

Emülatör algılamada yanlış pozitif riski neden önemli?

Cevap: Gerçek cihaz/geliştirici ortamı emülatör gibi görünebilir; sert tepki gerçek kullanıcıyı mağdur eder.

Soru 8

RASP'in temel sınırı nedir?

Cevap: Her tek denetim atlanabilir; köklü cihazda zayıflar. Güç katmandan, çeşitlilikten ve sunucu denetiminden gelir.

Quiz-1 tarzı örnek sorular

Örnek · çoktan seçmeli

RASP çapraz denetiminde amaç nedir?

- A) Kodu küçültmek
- B) İki tarafı da atlamayı zorunlu kılmak ✓
- C) Şifreleme hızını artırmak
- D) Günlüğü kapatmak

Örnek · kısa cevap

Bir bütünlük denetimini güçlendiren üç yöntem yazın.

Gizleme · çoklu/örtüşen blok · çapraz denetim · sonucu davranışa bağlama.

Sözlük

Terim	Anlam
RASP	Çalışma anı öz-koruma
Self-hashing	Kendi kodunun özetini denetleme
Hook/Frida	Fonksiyon çağrısını araya girip değiştirme
CFI sayacı	Denetim sırasını sayarak doğrulama
Cihaz bağlama	Veriyi belirli cihaza bağlama

Özet: bu haftanın tek cümlesi

RASP program çalışırken kendini izler ve tehdide tepki verir; ama tek denetim değil, **katmanlı, çeşitlendirilmiş, gizli** denetimler ve **sunucu tarafı** doğrulama ile güç kazanır.

Gelecek hafta

7. hafta — Ara proje gösterimi (RAP1) ve 8. hafta — Quiz-1

Vize dönemi: projenizin ilk yarısını gösterip Quiz-1'e hazırlanın.

Ek A · Bir RASP katmanı kurmak

Hedef

Sentetik bir "lisans denetimi"ni RASP ile koruyalım:

- bütünlük denetimi
- hata ayıklayıcı denetimi
- akıllı tepki

Adım 1 · korunacak nokta

```
int lisans_gecerli(void) {  
    /* ... kontrol ... */  
    return sonuc;    /* saldırganın hedefi */  
}
```

Bu fonksiyon ve onu çağıran akış kritik.

Adım 2 · bütünlük denetimi ekle

- Derleme sonrası bu bölgenin özetini kaydet.
- Çalışırken yeniden hesapla, karşılaştır.
- Fark → kurcalama bayrağı.

Adım 3 · anti-debug ekle

- Zamanlama + OS göstergesiyle "izleniyor muyum?".
- Sonucu tek başına bırakma; başka denetimle **birleştir**.

Adım 4 · sonucu davranışa bağla

- `lisans_gecerli` sonucu düz true/false değil.
- Bütünlük + anti-debug göstergeleri, sonucun **hesabına** girer.
- Kurcalama varsa fonksiyon **yanlış** çalışır.

Adım 5 · tepki

- Hemen çökme yerine: bayrağı sunucuya bildir, işlevi kısıtla.
- Gecikmeli tetik: kaynağı gizle.

Adım 6 · gizle ve çeşitlendir

- Denetim kodunu gizle (9. hafta).
- Her sürümde farklı yerleştirme (çeşitlendirme).
- Çapraz denetim ekle.

Adım 7 · ölç ve belgele

- Kaç denetim, nerede, hangi tepki?
- Başarım maliyeti (denetimler yavaşlatır).
- S10'a yaz.

Ek B · Mini vaka

Kurgu · ödeme uygulaması

- Yerel anahtar + ödeme akışı.
- Hedef: kurcalama ve canlı analiz zorlaştırsın.

Vaka · denetim yerleşimi

- **Başlangıç:** kök/emülatör/imza denetimi.
- **Ödeme öncesi:** bütünlük + anti-debug + hook denetimi.
- **Periyodik/rastgele:** tekrar.

Vaka · tepki

- Yüksek güvenli gösterge → işlemi reddet + sunucuya bildir.
- Zayıf gösterge → sunucu risk skorunu artır.
- Kullanıcı deneyimini koru (yanlış pozitif).

Vaka · katmanlar

- RASP + gizleme (9) + whitebox (11) + kısa ömürlü anahtar (10).
- Sunucu tarafı: hız sınırı, anomali.
- Bir katman aşılsa bile diğerleri sürer.

Vaka · kalan risk

- Köklü cihazda kararlı saldırgan ilerleyebilir.
- Ama: çok katman + sunucu denetimi → hasar sınırlı, saldırı pahalı.
- Açıkça yazılır.

Ek C · RASP denetim kataloğu

Katalog (1)

Denetim	Ne yakalar	Sınır
Bütünlük (self-hash)	İkili yama	Özet fn. atlanabilir
Anti-debug	Debugger bağlı	Bilinen yöntemler atlanır
Emülatör	Sahte ortam	Yanlış pozitif

Katalog (2)

Denetim	Ne yakalar	Sınır
Hook/Frida	Fonksiyon kancası	Gizli araçlar
Kök	Yetki yükseltme	Kök gizleyiciler
İmza	Sahte bileşen	Anahtar sızarsa
CFI sayacı	Denetim atlama	Karmaşık yama

Katalogdan ders

- Her denetimin bir **kör noktası** var.
- Tek başına hiçbirisi yeterli değil.
- Katalogdan **birçoğunu** seç, birleştir, çeşitlendir.

Denetim seçimi

- Varlığın değerine göre kaç katman?
- Platforma göre hangi denetimler anlamlı?
- Başarım bütçesi ne kadar?

Kararı ve gerekçesini S10'a yaz.

Ek D · Saldırgan $\leftrightarrow$ savunmacı

Neden bu diyalog?

Güvenlik bir **hamle-karşı hamle** oyunudur.

Her savunmaya bir saldırı, her saldırıya yeni savunma.

Katmanlı savunma bu yüzden gereklidir.

Bütünlük · diyalog

- **Savunma:** self-hashing ekle.
- **Saldırgan:** özet fonksiyonunu bul, hep "tutuyor" dedirt.
- **Savunma:** özeti gizle, örtüştür, sonucu davranışa bağla.

Anti-debug · diyalog

- **Savunma:** debugger denetimi ekle.
- **Saldırgan:** denetimi bul, atla.
- **Savunma:** çok yöntem + zamanlama + gizleme; tepkiyi gecikmeli yap.

Hook · diyalog

- **Savunma:** fonksiyon baytlarını denetle.
- **Saldırgan:** kancayı gizle, baytları geri düzelt.
- **Savunma:** çapraz denetim + sunucu tarafı doğrulama.

Kök · diyalog

- **Savunma:** kök göstergelerini denetle.
- **Saldırgan:** kök gizleyici kullan.
- **Savunma:** çok gösterge + sunucu risk skoru; sert tepki değil kısıtlama.

Diyalogdan ders

- Tek savunma her zaman aşılr.
- Güç: **çokluk + gizlilik + çeşitlilik + sunucu.**
- Amaç: saldırıyı **ekonomik olmaktan çıkarmak.**

Ek E · Tepki karar akışı

"Tehdit gördüm, ne yapayım?" — 1

Soru 1: Gösterge ne kadar güvenilir?

- **Çok güvenilir** → işlemi reddet + sunucuya bildir.
- **Belirsiz** → devam, ama risk skorunu artır.

"Ne yapayım?" — 2

Soru 2: Kullanıcı deneyimi etkilenir mi?

- Yanlış pozitif riski yüksekse → **sert tepki verme**.
- Sessiz kısıtlama + sunucu denetimi tercih.

"Ne yapayım?" — 3

Soru 3: Tepki denetimi ele verir mi?

- Hemen çökme → kaynağı gösterir.
- **Gecikmeli/dolaylı** tepki → kaynağı gizler.

Tepki akışı · tek bakış

Gösterge güvenilir mi?

evet -> reddet + sunucuya bildir (gecikmeli, dolaylı)

belirsiz -> devam + risk skoru artır

Her durumda: yanlış pozitif düşün, kaynağı gizle

Ek F · Hızlı başvuru

Sık yapılan hatalar

- Tek denetime güvenmek.
- Denetimi gizlememek (kolay bulunur).
- "Hemen çök" tepkisi (kaynağı ele verir).
- Sert tepki + yüksek yanlış pozitif (kullanıcı mağdur).
- Sunucu denetimini atlamak (asıl güvence orada).

RASP kontrol listesi

- [] ≥ 2 farklı denetim türü
- [] Denetimler gizli + çeşitlendirilmiş
- [] Çapraz denetim (Java $\leftrightarrow$ native)
- [] Akıllı tepki (gecikmeli/dolaylı)
- [] Cihaz bağlama
- [] Sunucu tarafı doğrulama
- [] Yanlış pozitif planı

Son söz (6. hafta)

RASP, saldırganla oynanan bir **zaman** oyunudur: her katman biraz daha zaman kazandırır.

Asıl güvence: katmanlı RASP + gizleme + kısa ömürlü anahtar + **sunucu denetimi**.